

# Tema 6 - Análisis Sintáctico Ascendente LR

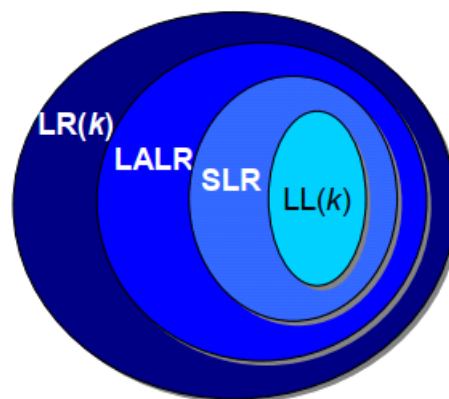
Profesor Ramón López-Cózar Delgado, Curso 21-22

## 6.1 Analizadores LR( $k$ ):

- Surgen como una alternativa a los problemas que presentan las restricciones gramáticas LL( $n$ ) para análisis descendiente y las gramáticas de precedencia en análisis ascendente.
- En Precedencia, solo se mira el símbolo que está en el tope de la pila con el símbolo de entrada, sin tomar en cuenta los anteriores en la pila, esto puede tener como consecuencia que no se forme una secuencia correcta de símbolos para reducir.
- Son el método más utilizado y más general.
  - **L** porque analiza los símbolos de entrada de izquierda a derecha, (left-right).
  - **R** porque reduce a la derecha (right).
  - **$k$**  por el número de símbolos de anticipación.
- Existen tres técnicas para la confección de la tabla de análisis y cada una de ellas caracteriza tres tipos de gramáticas LR que son
  - **LR Simple (SLR)**: La más fácil de implantar pero la menos potente.
  - **LR Canónica o LR(1)**: La más potente pero la más compleja.
  - **LALR (LR con anticipación, o look-ahead)**: Es la técnica intermedia y más usada, se trata de una simplificación del método general LR(1) y usado en las implementaciones de generadores sintácticos basados en YACC.
- Esta técnica es válida para gramáticas libre de contexto no ambiguas.

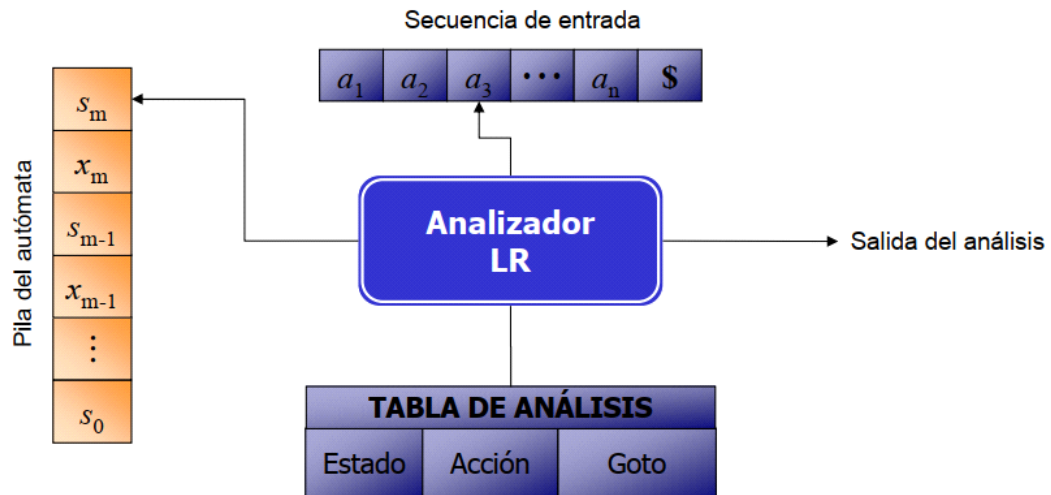
### **Ventajas de los Analizadores LR( $k$ ):**

- Puede reconocer virtualmente todos los lenguajes que obedecen a una gramática libre de contexto
- Es el método conocido más general sin retroceso, así como el más fácil de implementar de los conocidos Reducción y Desplazamiento.
- Las clases de gramáticas que pueden analizarse mediante el analizador LR son el superconjunto de las gramáticas que pueden analizarse por analizadores predictivos.
- El analizador LR puede detectar errores conforme se realiza el análisis, así como aplicar recuperación del análisis con pérdida controlada: es decir, se puede evitar una avalancha de ir quitando elementos del tope de la pila.

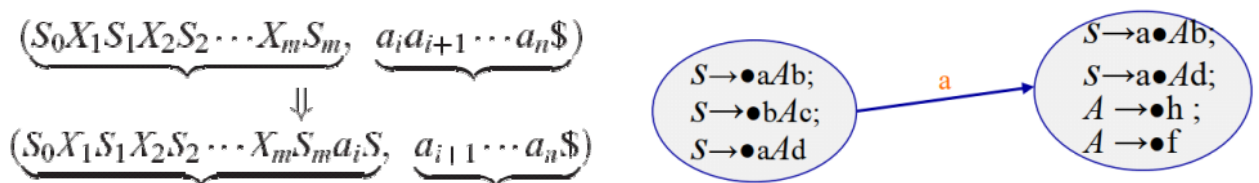


## 6.2 - Descripción Funcional:

- La pila del autómata utilizado para el análisis LR está formado por parejas de símbolos (terminales y no terminales) y estados.



- Si  $acción[S_m, a_i] = \text{Desplaza}$ , el analizador inserta en la pila tanto el símbolo de entrada  $a_i$  como el siguiente estado  $S$  que se obtiene fruto de la evaluación  $goto(S_m, a)$ , ahora  $a_{i+1}$  pasa a ser el símbolo de entrada a leer y el analizador se encuentra ahora en el estado  $S$



- Si  $acción[S_m, a_i] = \text{Reduce}$ , entonces el analizador ejecuta una reducción utilizando la regla  $A \rightarrow \beta$
- Si  $acción[S_m, a_i] = \text{Acepta}$ , entonces el análisis ha terminado con éxito.
- Si  $acción[S_m, a_i] = \text{Error}$ , se ha descubierto un error, el analizador llamará entonces al procedimiento de recuperación de errores.

#### Cálculo de Iniciales:

- Recorrer las producciones y notar cuales son los símbolos que aparecen de primero, si el primer símbolo es un no terminal, se añaden los iniciales de ese, si también desencadena en  $\epsilon$  entonces se añaden iniciales del segundo símbolo quitando la cadena vacía.
- Si no hay segundo símbolo, entonces se queda añadida la cadena vacía.

#### Cálculo de Seguidores:

- Si el símbolo es el inicial, automáticamente \$ es parte de los Seguidores.
- Si el símbolo siguiente es terminal, se añade.
- Si no es terminal, se añaden los Iniciales de ese símbolo menos la cadena vacía
  - Si tiene la cadena vacía, se añaden Seguidores del símbolo que produce la producción.
- Si el símbolo a analizar es el último, se añaden Seguidores del símbolo que produce la producción sin la cadena vacía.

### 6.3 Método de construcción de una tabla SLR:

- **Pivote:** Subcadena que coincide con la parte derecha de una producción
- **Prefijo viable:** Sea una cadena  $\phi\beta t$  y  $\beta$  es subcadena pivote, y  $t$  un símbolo terminal, se define prefijo viable como la secuencia de símbolos que se pueden formar de izquierda a derecha hasta el símbolo  $t$ .
- **Ítems o configuración de la gramática:** Será la producción representada entre  $[ , ]$  y con un "." en la parte derecha de la producción
- **Objetivo:** Construir un autómata finito determinista que reconozca los prefijos viables a partir de una gramática, los estados del autómata finito determinista se obtienen en base a agrupar los ítems

asociados en cada producción de la gramática.

### Construcción del AFD para SLR

- Se aumenta la gramática, sea  $S$  el símbolo inicial de una gramática  $G$ , se define una gramática  $G'$  llamada aumentada tal que el símbolo inicial de la gramática es  $S'$  y aparece la producción  $S' \rightarrow S$
- Operación de clausura de un conjunto de ítems de una gramática: Sea  $I$  el conjunto de ítems, la clausura de  $I$ , que se nota como  $clausura(I)$  será otro conjunto de ítems obtenidos por las reglas siguientes:

- $\{I\} \subset \{clausura(I)\}$ ,
- Si  $[A \rightarrow \alpha \cdot B \gamma]$  y existe la producción  $B \rightarrow \beta$ , entonces  $[B \rightarrow \cdot \beta]$  y  $[A \rightarrow \alpha \cdot B \gamma] \in \{clausura(I)\}$ .

Ejemplo 6.3: Sean las producciones siguientes:

$$E \rightarrow E; E \rightarrow E + T \mid T; T \rightarrow T * F \mid F; F \rightarrow (E) \mid id$$

$$I = \{ [E \rightarrow \cdot E] \}$$

$$clausura(I) = \{ [E \rightarrow \cdot E], [E \rightarrow \cdot E + T], [E \rightarrow \cdot T], [T \rightarrow \cdot T + F], [T \rightarrow \cdot F], [F \rightarrow \cdot (E)], [F \rightarrow \cdot id] \}.$$

- Función goto:** Es una función de transición de estados alcanzables a partir de uno dado y según el símbolo que haya en la entrada

Sea  $I$  un conjunto de ítems, sea  $X$  un símbolo cualquiera de la gramática. Se define la función **goto** del conjunto de ítems  $I$  dado un símbolo  $X$  de la gramática, notado como  $goto[I, X]$ , como la clausura de ítems del tipo:

$$[A \rightarrow \alpha X \cdot \beta] \text{ tal que } [A \rightarrow \alpha X \cdot \beta] \in I.$$

Ejemplo 6.4: Dada la gramática aumentada tomada del ejemplo anterior.

$$\begin{array}{l} E' \rightarrow E \\ E \rightarrow E + T \mid T \\ T \rightarrow T * F \mid F \\ F \rightarrow (E) \mid id \end{array}$$

y sea el conjunto de ítems siguiente:

$$I = \{ [E' \rightarrow E \cdot], [E \rightarrow E \cdot + T] \}$$

Entonces:

$$goto[I, +] = \{ [E \rightarrow E + \cdot T] \}$$

Posteriormente se debe obtener el siguiente estado que se alcanza. Para ello calculamos la función  $clausura(goto[I, +])$  que resultaría:

$$clausura([E \rightarrow E + \cdot T]) = \{ [E \rightarrow E + \cdot T], [T \rightarrow \cdot T * F], [T \rightarrow \cdot F], [F \rightarrow \cdot (E)], [F \rightarrow \cdot id] \}$$

## Estados asociados con la gramática SLR.

### Proceso de obtención:

- El estado inicial  $I_0$  se define como la  $clausura([S' \rightarrow \cdot S])$  donde  $S$  es el símbolo inicial de la gramática.
- Los estados sucesivos se obtiene como  $I_n = goto(I_{n-1}, N)$  para todo símbolo terminal o no terminal con el punto delante que aparecen en los ítems en  $I_{n-1}$ .
- Se aplica el paso (2) hasta que no se generen nuevos estados.

### Conflictos en la tabla de análisis SLR

- En gramáticas SLR, las reducciones se aplican ante cualquier símbolo seguidor del símbolo no terminal, esto en algunos casos podría producir conflictos, si esto es así la gramática no es SLR.
- Otro problema que surge en SLR es el ejemplo de sentencias para llamada a procedimiento y la sentencia de asignación. En ambos casos se comienza con un identificador. Hasta llegar al siguiente

token (en este caso la asignación "=") no se sabría aplicar la acción adecuada.

#### 6.4 Construcción de tabla de análisis LR(1):

- El proceso de análisis LR(1) significa llevar un símbolo de anticipación para decidir el análisis, se extiende el concepto de ítem, el cual incluye un símbolo de anticipación

**Ítems con anticipación:**

Será de la forma  $[A \rightarrow \alpha \cdot \beta, a]$ , donde  $a$  es un símbolo terminal de la gramática.

- Significado:* Sea  $[A \rightarrow \alpha \cdot, a]$ . Se reducirá mediante  $A \rightarrow \alpha$  si sólo si, el siguiente símbolo de entrada es  $a$ .

**Validez de los ítems**

- El ítem  $[A \rightarrow \alpha \cdot \beta, a]$  es válido para el prefijo  $\delta\alpha$  si  $S \Rightarrow^* \delta A \omega \Rightarrow \delta \alpha \beta \omega$ ; y  $\omega \Rightarrow a \omega'$  y  $a \in \text{Iniciales}(\omega)$  en el caso que  $\omega = \epsilon$   $S \Rightarrow^* \delta \alpha \beta \$$

**Clausura**

- $I_0$  es la clausura( $[S' \rightarrow \cdot S, \$]$ ).
- Sea el ítem  $[A \rightarrow \alpha \cdot B \beta, a]$  y la producción  $B \rightarrow \eta$ , entonces habrá que incluir en la clausura el ítem  $[B \rightarrow \cdot \eta, b]$  donde  $b \in \text{Iniciales}(\beta a)$ .

**Justificación de la regla (2):**

Si  $\delta\alpha$  es prefijo válido de  $[A \rightarrow \alpha \cdot B \beta, a]$  implica  $S \Rightarrow^* \delta A \omega \Rightarrow \delta \alpha B \beta \omega$  donde  $a \in \text{Iniciales}(\omega)$ , también  $\delta\alpha\eta$  es prefijo válido de  $[B \rightarrow \cdot \eta, b]$  si  $\delta\alpha B \beta \Rightarrow \delta\alpha\eta\beta$  donde  $b \in \text{Iniciales}(\beta a)$ .

- Los  $\text{INICIALES}(\beta a)$  son muy importantes, porque nos van a decir cuales son los símbolos de anticipación,  $\beta$  es el símbolo que está detrás del símbolo que tiene el punto en frente.
  - Si se tiene  $A \rightarrow ab \cdot xy$ ,  $\beta$  es  $y$ .

#### 6.5 Construcción de la tabla LALR(1):

**Algoritmo para construir la tabla LALR(1)**

**Entrada:** La gramática aumentada.

**Salida:** La funciones *acción* y *goto*.

**Método:**

- Construir el conjunto de estados como si fuese LR(1).
- Unir los estados formados por los mismos ítems con independencia de los símbolos de anticipación en los ítems.
- Considerar la presencia del estado  $j$  unido con  $ikh$  como el nuevo estado  $jikh$ .

#### 6.6 Tratamiento de la ambigüedad:

- Todos los métodos de análisis sintáctico predictivo requieren que la gramática no sea ambigua, sin

embargo, hay algunos casos de conflicto que se pueden resolver adoptando algún criterio o base a información adicional sobre el lenguaje que no se refleja en su gramática, como los operadores.

#### **Conflictos reduce/desplaza**

- Si se opta por reducir, significa que se está aceptando como válida la secuencia. Se debe asegurar que en esa secuencia inicialmente aceptada, se alcance el símbolo inicial de la gramática en el caso que la secuencia fuese correcta.
- Si se opta por desplazar, se actúa de forma contraria. Así es como actúa YACC.

#### **Conflictos reduce/reduce**

- Surgen por tener reglas redundantes, y se resuelven fácilmente eliminando aquellas producciones equivalentes.

#### **Ambigüedad entre operadores**

- La precedencia y la asociatividad de los operadores nos aportan criterios para eliminar los conflictos que puedan aparecer en la tabla de análisis.
- Cuando en una celda de la tabla de análisis aparecen conflictos desplaza/reduce bajo un mismo símbolo operador, se actuará de la siguiente forma cuando se defina precedencia y asociatividad entre los operadores
  - Si  $op_1 < op_2$  entonces en  $op_2$  debe aparecer **desplaza** y en  $op_1$  **reduce**.
    - Por ejemplo, si fuera multiplicar con suma, entonces en donde aparece \* se coloca desplazar y en + la reducción.
  - Si en una producción aparece reducir, eso es asociatividad por la izquierda, si es desplazar es por la derecha.

#### **Ambigüedad de los "else-balanceados"**

- Cuando en la gramática se dispone de la sentencia if-then-else (sin usar endif) se produce un conflicto desplaza/reduce, la solución que se adopta (y adopta YACC) es tomar la asociatividad del else con el último then, de este modo se pasa el else a la pila y se desplaza.

### **6.7 Detección y recuperación de errores:**

- La técnica para detectar errores y recuperación de errores en gramáticas LR se basa en acotar el contexto donde se ha producido el error
  - Modo pánico (Panic)
  - Tabla de análisis
  - Métodos complejos basado en el algoritmo de Graham-Rhodes

#### **Modo pánico**

- Se saltan símbolos de la entrada y/o sacan símbolos de la pila hasta alcanzar una situación válida para continuar el análisis, se elimina el error y más información de entrada.
- Se quitan cosas de la pila para seguir llevando el análisis sintáctico, aunque en la pila estén sintácticamente correctos, se van quitando símbolos de la pila hasta llegar a un símbolo de sincronización y a partir de ahí se consigue el análisis.
- Un problema es que si se omiten algunos símbolos de sincronización, se pueden quitar demasiados símbolos de entrada.
  - La solución es optimizar los símbolos de sincronización en base a los símbolos seguidores e iniciales de cada símbolo no terminal de la gramática

#### **Tabla de análisis**

- Consiste en completar las celdas vacías de la tabla de análisis con procedimientos concretos para el tratamiento y recuperación de errores de forma que se pueda continuar el análisis

| Estado | Acción |    |     | Goto |   |
|--------|--------|----|-----|------|---|
|        | a      | d  | S   | S    | C |
| 0      | s3     | s4 |     | 1    | 2 |
| 1      |        |    | ACC |      |   |
| 2      | s6     | s7 |     |      |   |
| 3      | s3     | s4 |     |      | 8 |
| 4      | r3     | r3 |     |      |   |
| 5      |        |    | r1  |      |   |
| 6      | s6     | s7 |     |      | 9 |
| 7      |        |    | r3  |      |   |
| 8      | r2     | r2 |     |      |   |
| 9      |        |    | r2  |      |   |

Si el analizador se encontrase en esta situación, podría realizar las siguientes tareas:

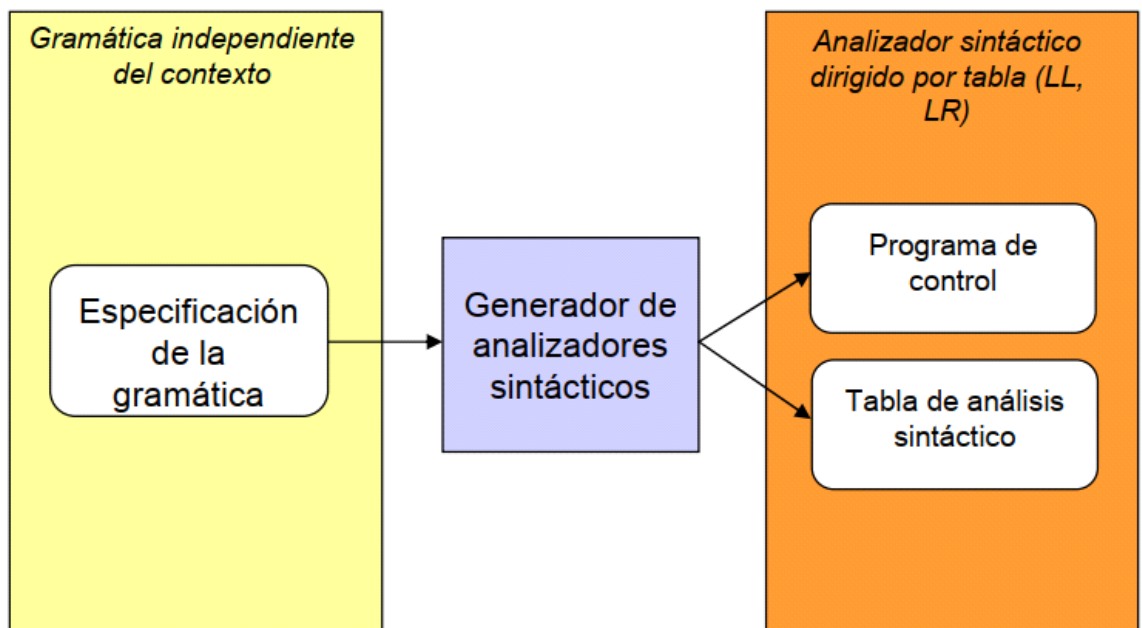
- Informar del error diciendo: esperaba "a" o "d" (...que son los símbolos de entrada con transición válida en la tabla).
- Localizar aquellos símbolos que están en la pila y que rodean el error (sacarlos de la pila)
- Saltar símbolos de la entrada hasta poder continuar con el análisis.

### Métodos complejos basado en el algoritmo de Graham-Rhodes

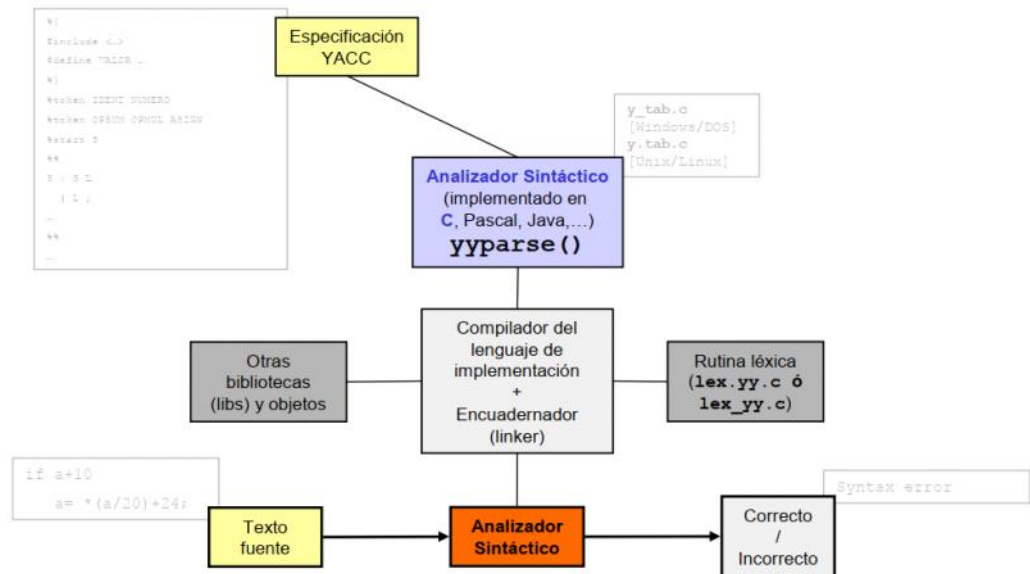
- Permite recuperarse ante un error y además, en algunos casos, reparar el mismo.
- El método tiene dos fases:
  - **Fase de condensación:** Se acota el error del contexto
  - **Fase de corrección:** Se intenta corregir el error
- Basándose en el análisis de símbolos próximos en el tope de la pila, y en la entrada, la fase de corrección decide si hay que sacar de la pila o saltar en la entrada, actuando como si hubiese insertado o ignorado símbolos de la entrada.

### 6.8 YACC:

- Es un generador de analizadores sintácticos
  - Hace uso de análisis sintáctico ascendente basado en gramáticas LALR(1)
  - Requiere una rutina léxica yylex() para reconocer los tokens, usando LEX.



## 6.8.1 Entorno de YACC



**yacc** [opciones] nombre\_del\_fichero\_de\_especificación.

- v genera la tabla de análisis sintáctico en el fichero y.out.
- b prefijo permite cambiar el prefijo y del nombre del fichero de salida por el especificado.
- t permite realizar un "debugging" del análisis sintáctico.
- d genera el fichero de cabecera y\_tab.h, que contiene la definición de los símbolos (tokens).
- r genera dos ficheros de salida, y\_code.c que contiene el código y y\_tab.c que contiene la tabla de análisis.

- **Analizador léxico:** Para realizar la función de analizador léxico, YACC usa una función llamada yylex() que devuelve un entero, cuyo valor se asocia a la variable yyval
  - Este analizador puede ser:
    - Una función generada por LEX
    - Una función definida por el usuario en la zona de procedimientos de usuario

## Estructura de un fichero de especificación de YACC

```
{Declaraciones de símbolos}

%%

{Reglas gramaticales} + {Acciones}

%%

{Procedimientos del usuario}
```

Se pueden incluir comentarios con la siguiente sintaxis:

```
/* ejemplo de comentario */
```

```
/* ejemplo de comentario */
```

## Declaraciones de símbolos

En esta zona tenemos:

- **Código en C** opcional (Ejemplo: declaraciones de variables e inclusión de ficheros).

```
%{  
    /* código C */  
%}
```

- Definición del **símbolo inicial** de la gramática:

```
%start simbolo_inicial
```

- Declaraciones de todos los **nombres de símbolos** (terminales) que utiliza el lenguaje, para ello podemos usar:

- Para todos los nombres de los símbolos (terminales):

```
%token nombre_simbolo1 nombre_simbolo2 ... nombre_simbolo_n
```

Ejemplo:

```
%token IDENTIFICADOR NUMERO  
%token OP_SUMA ASIGNACION
```

- Para los nombres de los símbolos que representan operadores:

```
%left OP      /* asociatividad a la izquierda  (X OP Y) OP Z */  
%right OP     /* asociatividad a la derecha   X OP (Y OP Z) */  
%nonassoc OP  /* sin asociatividad    X OP Y OP Z = error */
```

## Reglas gramaticales

El formato general de una regla gramatical es:

**A** : cuerpo ;

Donde **A** es el nombre de un símbolo no terminal (o estructura sintáctica), y **cuerpo** es una secuencia de nombres de símbolos terminales y/o no terminales y/o literales, que indican la estructura del símbolo no terminal que se define.

Ejemplo:

```
A : B C D ;  
A : E F ;  
A : G ;
```

Cuando un símbolo no terminal dispone de varias alternativas, como el caso del ejemplo anterior, es posible definirlo de la siguiente forma:

Ejemplo:

```
A : B C D ;  
  | E F ;  
  | G ;
```

## Tratamiento de la ambigüedad

Se dice que una gramática es **ambigua** cuando alguna de sus sentencias (conjuntos de símbolos) puede ser estructurada de distintas formas.

Si la gramática es ambigua, YACC nos informará en los siguientes términos:

- Conflicto **desplaza/reduce** (**shift/reduce conflict**).
- Conflicto **reduce/reduce** (**reduce/reduce conflict**).

Las posibles soluciones:

- Introducir **asociatividad** y **precedencia** entre los **operadores**.
- Estudiar la **tabla de análisis** en detalle y **modificarla**.

### Tratamiento de errores

- **Tratamiento por defecto:** En una situación de error se produce una llamada a la función `yyerror` que imprime un mensaje por pantalla `syntax error` y finaliza la ejecución del analizador sintáctico.
- **Introducción de producciones de error:** En la especificación YACC se pueden introducir producciones de error con el siguiente formato:  
$$A \rightarrow error\ w \text{ o } A \rightarrow error$$
  - $A$  es un símbolo no terminal de la gramática,  $error$  es una palabra reservada de YACC y  $w$  un símbolo terminal.
- YACC generará una tabla de análisis a partir de la especificación, dando producciones de error el mismo tratamiento que al resto de producciones.

### Actuación de YACC ante errores

- Sea  $S$  el estado del tope de la pila
  - Si para  $S$  existe un desplazamiento con el símbolo de  $error$  lo aplicará
  - Si para  $S$  no existe un desplazamiento con el símbolo de  $error$ , extrae símbolos de la pila hasta que encuentra un estado que incluya un ítem de la forma  $A \rightarrow \cdot error\ w$ , introduce  $error$  en la pila.
    - Si  $w$  es distinto de la cadena vacía, eliminará caracteres de entrada hasta encontrar  $w$  y se reducirá con la producción de error.
    - Si  $w$  es la cadena vacía, se pasa a reducir por la producción de error cuando acontezca algún seguidor de  $A$ .
      - Forma recomendada, que YACC meta los símbolos de sincronización. YACC calcula los seguidores y esos son los símbolos de sincronización